

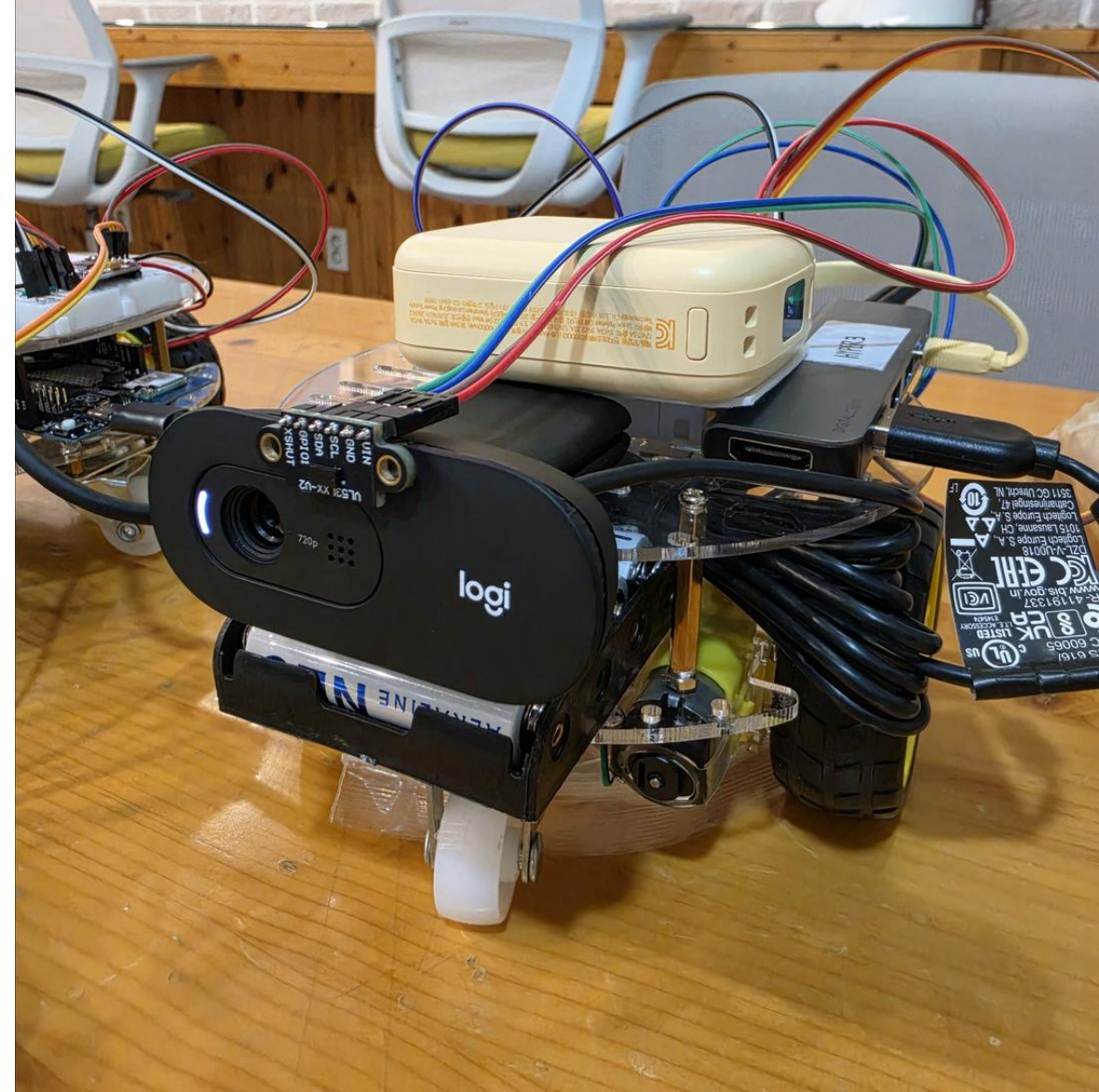
2026-2 HYPER 자율주행트랙

Q-ROVER

2차시

센서와 비전을 이용한
장애물 감지

9/21 (월) 18:30-20:00



HYPER

OPENING 오늘의 목차

01	거리센서	배선하고 거리값 읽기
02	Bridge / RPC	MCU와 Python 사이에 값 전달
03	리눅스 실습	SSH, 파일 탐색, 5인 조사, 원격 화면
04	카메라와 비전	픽셀, OpenCV, 객체 검출
05	객체인식 예제	실행, 데이터 경로, 검출 유지 시간
06	로버 통합	거리 또는 병 검출 시 정지

중간 중간에 계속 실습이 준비되어 있습니다!

OPENING 지난 시간과 오늘



1차시 · 정해 둔 시간만큼 주행

delay(1000) 동안 전진
장애물 판단 없이 MCU만 사용

Q-ROVER 연결됨

카메라
모델 4912 - 모델 컨테이너가 적절 보냅니다

chair (0.77)
chair (0.67)

거리
VL53L0X - MCU
554 mm
정지 기준 200 mm

로버 상태
직진
거리센서 통과 MCU가 판단
카메라 통과 AI가 판단

Confidence 임계값
0.50
낮추면 오검출↑ · 높이면 놓침↓
되돌리기

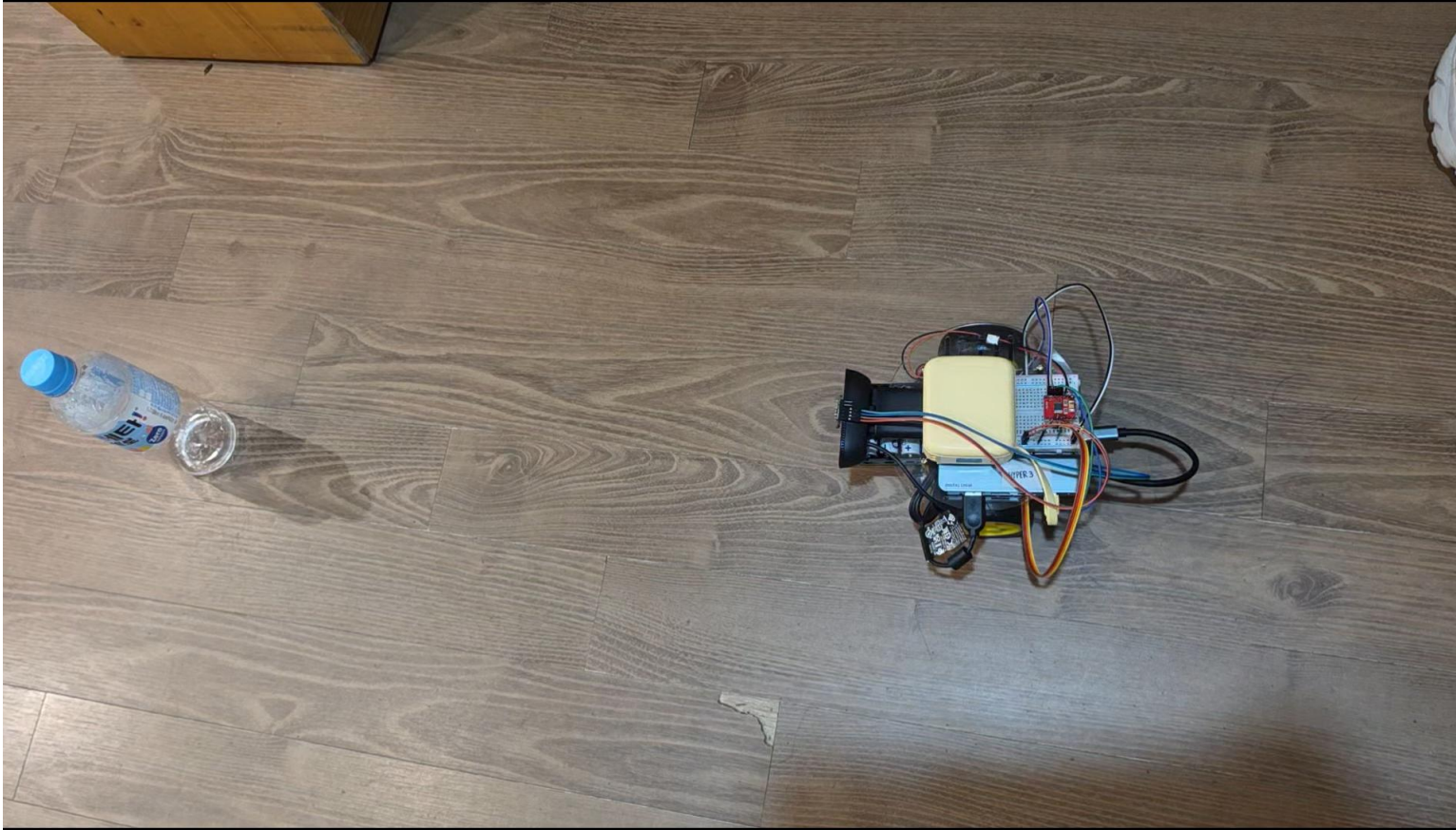
최근 검출

chair	67%	오후 11:52:58
chair	76.5%	오후 11:52:58
chair	54%	오후 11:52:57
chair	72.8%	오후 11:52:57
chair	56.7%	오후 11:52:57

2차시 · 보고 판단해서 주행

거리센서와 카메라로 정지 조건 판단
MCU와 MPU가 함께 동작

OPENING 오늘의 목표



01 직진

02 가까이 막히면 정지

03 병이 보이면 정지

04 치우면 재출발

거리센서 정지와 카메라 정지를 동시에 판단하는 구조

MCU와 MPU · 보드 안의 두 두뇌

UNO Q 한 장에는 성격이 다른 컴퓨터 두 개가 함께 들어 있습니다

MCU STM32U585

Micro Controller Unit

비유 반사 신경 · 정해진 일을 빠르고 꾸준히 하게

일상 속 예	전자레인지 · 세탁기 · 리모컨 속 칩
일하는 방식	켜자마자 한 프로그램을 끝없이 반복
메모리	KB 단위 (786 KB)
오늘 맡은 일	거리센서 읽기 · 모터 정지/주행

MPU QRB2210

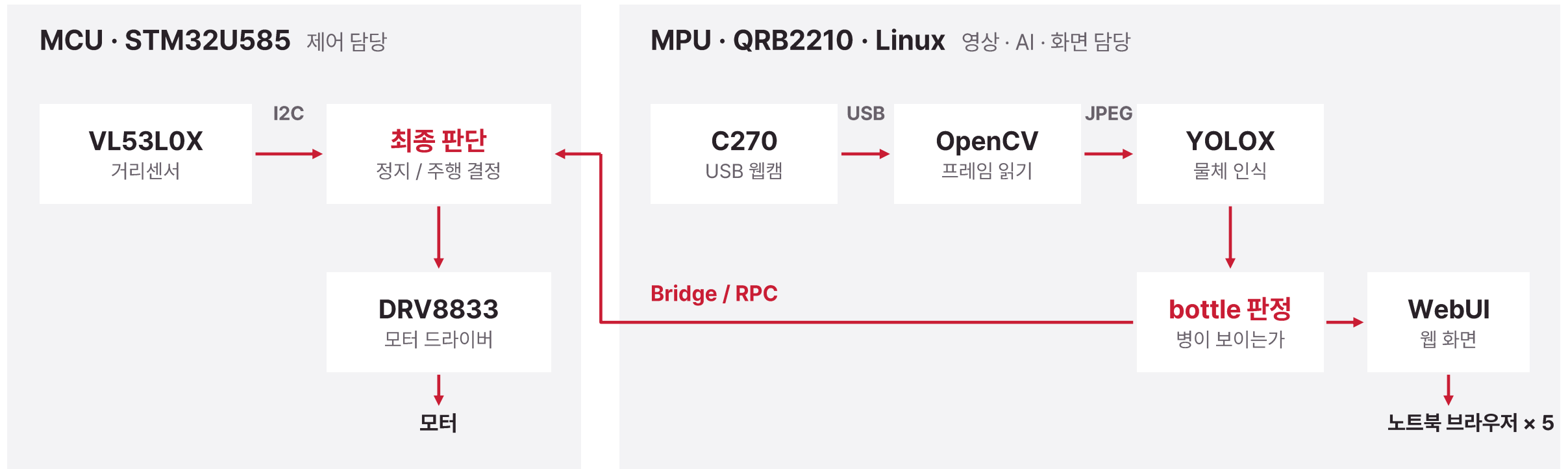
Micro Processor Unit

비유 생각하는 뇌 · 복잡한 일을 여러 개 동시에

일상 속 예	스마트폰 · 노트북
일하는 방식	리눅스 위에서 여러 프로그램을 동시에
메모리	GB 단위
오늘 맡은 일	카메라 영상 · AI 물체 인식 · 웹 화면

뜨거운 걸 만지면 생각보다 손이 먼저 떨어지듯, '멈춤' 결정은 빠른 MCU가 말합니다

OPENING 오늘의 데이터 흐름

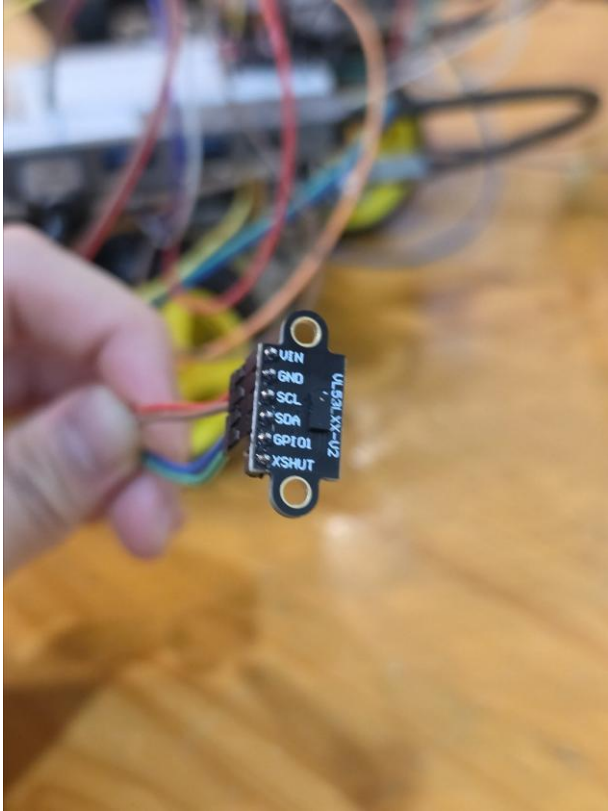


거리 판단과 최종 제어는 MCU, 영상 인식과 화면은 MPU

Part 1

적외선 센서와 통신 방식

[개념] VL53L0X(적외선 거리센서): 빛으로 거리를 측정한다!



ToF · Time of Flight

센서

940 nm 적외선 왕복 시간 t

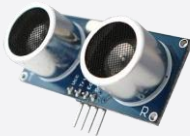
물체

$$d = c \times t / 2$$

1 mm 왕복은 약 6.7 ps

최대 약 2 m, 시야각 약 25°

검은색·투명·반사 물체는 측정이 어려울 수 있음



초음파 센서와의 차이??

초음파는 소리의 왕복 시간을 측정합니다.

검출 범위와 속도는 센서·설정·대상에 따라 달라집니다.

[개념] 보드 내부의 통신 방식: 센서와 대화하는 법

UART

I2C · 오늘 사용

SPI



	UART	I2C	SPI
신호선	TX/RX 2선	SDA/SCL 2선	3선 공유 + 장치별 CS
장치 구분	보통 1:1	주소로 구분	CS로 선택
주요 용도	GPS·디버그	센서 여러 개	SD카드·디스플레이

[개념] 코드 읽는 법 · 아두이노 스케치 기초

오늘 코드를 따라가는 데 필요한 문법은 이 정도면 충분합니다

```
#include <Wire.h>           // 라이브러리 불러오기
int distance_mm = -1;       // 변수 만들기
void setup() {              // 켜질 때 딱 1번
    Wire.begin();           // 함수 호출
}
void loop() {               // 끝없이 반복
    if (distance_mm < 200) { // 조건이 참이면
        brake();           // { } 안을 실행
    }
    delay(100);             // 100 ms(0.1초) 쉬기
}
```

setup() 과 loop()

전원이 켜지면 setup을 한 번, 그다음 loop를 끝없이 반복합니다.

변수

값을 담아 두는 이름표 상자. int는 정수, bool은 참/거짓을 담습니다.

함수 호출 · 이름()

Wire.begin()은 'Wire 도구의 begin 기능을 실행해'라는 뜻입니다.

라이브러리

남이 만들어 둔 기능 묶음. 센서를 다루는 복잡한 코드를 대신해 줍니다.

기호

// 뒤는 설명(주석), ; 은 문장 끝, { } 는 묶음의 시작과 끝

코드는 위에서 아래로 한 줄씩 실행되고, loop()의 끝에 닿으면 다시 처음으로 돌아갑니다

PART 1

실습 1

MCU에서 센서값 확인

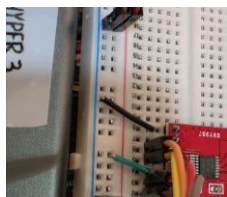
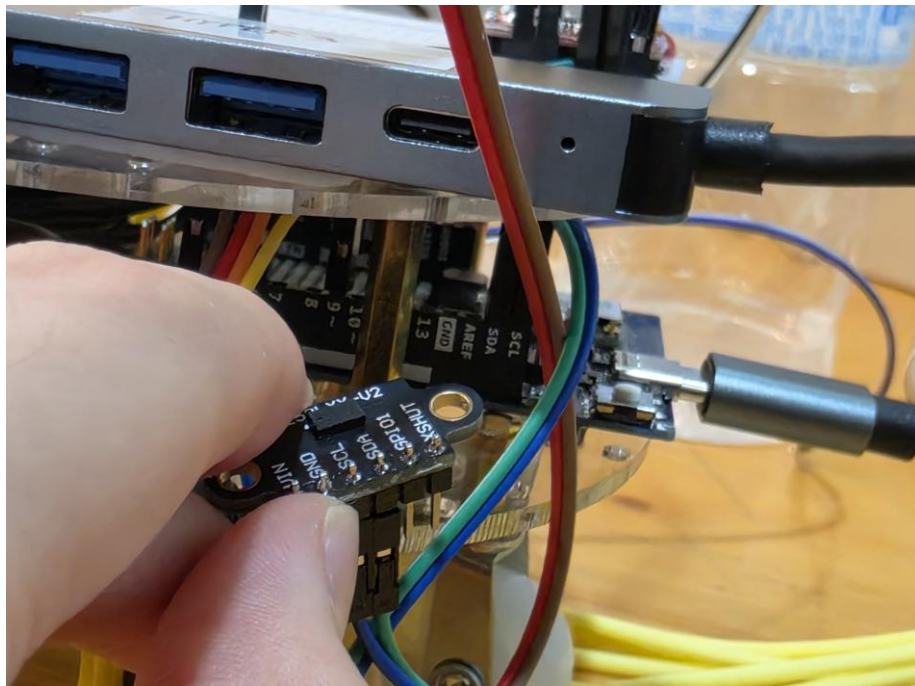
거리센서가 MCU까지 제대로 연결됐는지 확인하는 단계입니다.

App Lab에서 Serial Monitor 탭을 열면 거리센서가 I2C로 읽은 거리값(mm)이 계속 찍힙니다.

손을 가까이 대서 숫자가 줄어들면 성공입니다!

먼저 끝난 팀은 검은 물체, 투명 컵, 비스듬한 면을 비교하면서
측정 가능한 최소·최대 거리를 기록합니다.

PART 1 [실습 1] 거리센서 배선



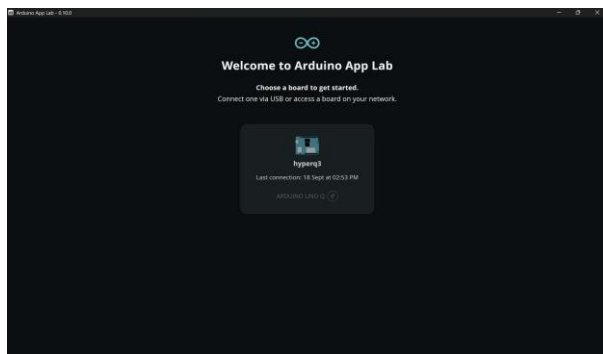
신호는 전압차입니다.
GND가 같아야
기준점도 같습니다.

핀	선의 색깔	UNO Q
VIN	빨강	모듈 허용 전원(2.6V~5V)
GND	갈색	GND
SCL	파랑	D21 / SCL
SDA	초록	D20 / SDA
GPIO1	미사용	연결 안 함
XSHUT	미사용	연결 안 함

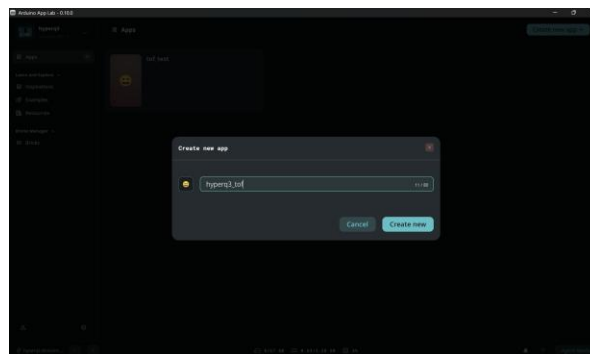
코드에서 사용될 `Wire.begin()`이 기본 I2C 핀을 사용합니다.
모듈의 전원·신호 전압을 확인하고 연결합니다.

VL53L0X(거리센서) 기본 주소는 0x29 · 같은 센서 여러 개는 XSHUT로 순차 설정

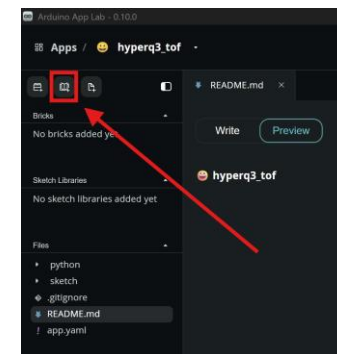
PART 1 [실습 1] App Lab 앱과 라이브러리



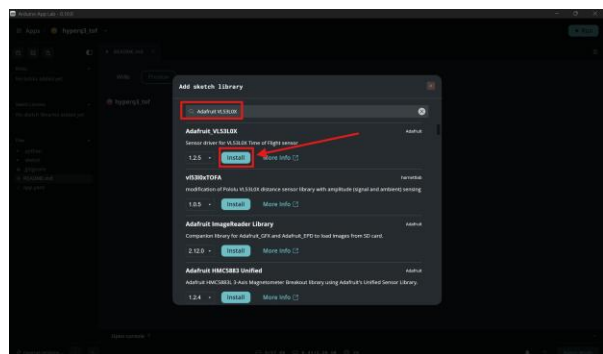
1 UNO Q 유선 연결한다



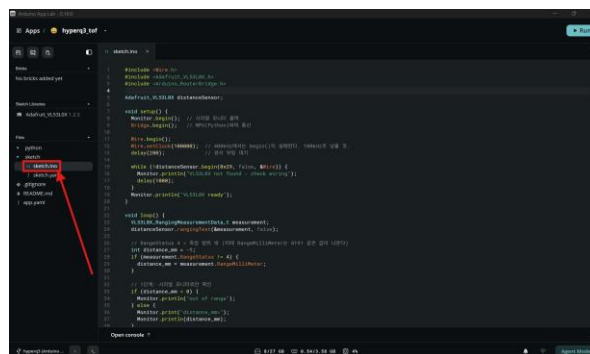
2 새 앱을 생성한다



3 Libraries를 열다



4 Adafruit VL53L0X를 추가한다



5 sketch.ino에 들어간다

sketch/sketch.yaml
라이브러리 의존성 목록
코드와 함께 전달

PART 1 [실습 1] 코드 ① · 센서 읽기

깃허브에서 코드 ①을 sketch.ino() 파일에 복사 + 붙여넣기 해줍니다. 다음은 코드 ①의 일부입니다.

```
// setup() 핵심
Monitor.begin();
Bridge.begin();
Wire.begin();
Wire.setClock(100000);
delay(200);
while (!distanceSensor.begin(
    0x29, false, &Wire)) {
    delay(1000);
}
```

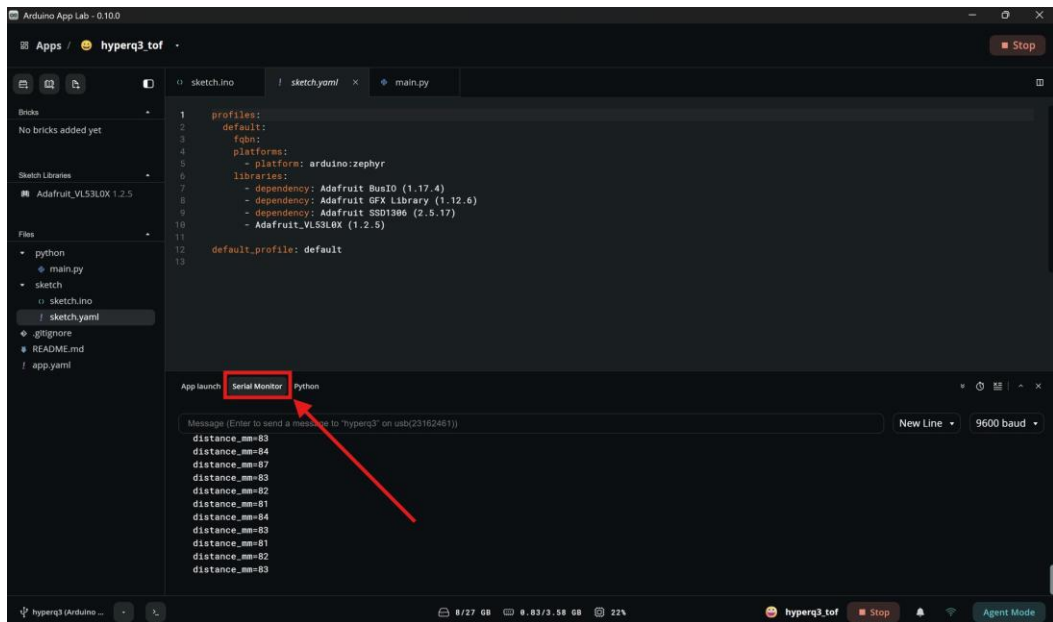
100 kHz: 수업 배선에서 확인한 설정
200 ms: 센서 시작 전 대기

```
// loop() 핵심
VL53L0X_RangingMeasurementData_t m;
distanceSensor.rangingTest(&m, false);
int distance_mm = -1;
if (m.RangeStatus != 4) {
    distance_mm = m.RangeMilliMeter;
}
Monitor.println(distance_mm);
delay(100);
```

0x29: 센서의 I2C 주소
상태 4일 때 -1로 변환하는 수업용 코드

-1은 유효한 거리값이 없다는 표시입니다. 실제로 멀다는 보장은 없습니다.

[실습 1] MCU에서 센서값 확인



코드 ① 실행 후 Serial Monitor 탭 열어줍니다.
Bridge.notify(...)는 아직 주석으로 유지합니다.

먼저 끝났다면
검은 물체, 투명 컵, 비스듬한 면을 비교하고
측정 가능한 최소·최대 거리를 기록합니다.

MCU 시리얼 모니터

I2C로 받은 거리값이 반복 출력됩니다.
손을 가까이 대면 숫자가 줄어듭니다.

증상

해결 방법

센서 not found

전원·GND와 SDA/SCL 재연결
I2C를 100 kHz로 설정

out of range만
표시

가까운 불투명 물체로 시험
렌즈 보호필름 제거

값이 불안정

직사광을 피하고 렌즈 청소
느슨한 배선 고정

Part 2

MCU와 MPU의 연결 방식, Bridge

PART 2

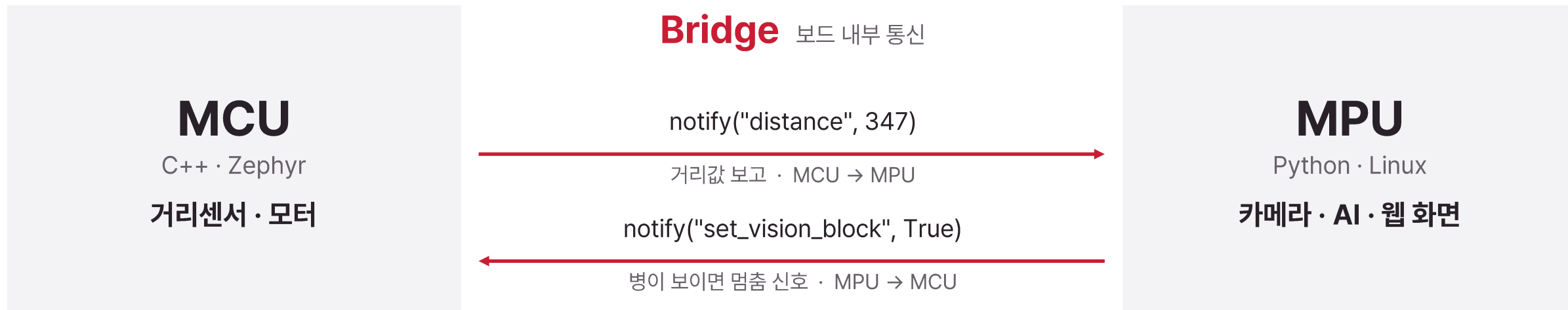
[개념] 두 프로세서의 역할

	MCU · STM32U585	MPU · QRB2210
실행 환경(운영체제)	Zephyr RTOS	Debian Linux
메모리	786 KB SRAM	GB 단위 LPDDR4
주요 역할	GPIO, 센서, 모터 PWM	영상, AI, 네트워크
오늘의 책임	거리 조건 + 최종 정지	bottle 판정 + WebUI

리눅스의 처리 지연과 별개로 거리센서를 읽고 멈추도록, 최종 제어를 MCU에 둡니다

PART 2 Bridge의 역할 · 두 두뇌를 잇는 내부 통로

MCU와 MPU는 한 보드에 있지만, 메모리도 언어도 따로인 두 컴퓨터입니다. 이 사이를 Bridge로 연결해줘야합니다!



따로 사는 두 컴퓨터

Python은 MCU의 변수를 직접 읽을 수 없습니다. 값을 메시지로 보내야 합니다.

이름으로 부르기

provide로 등록한 이름으로 값을 보냅니다. 양쪽 이름이 같아야 연결됩니다.

Wi-Fi와는 다른 길

Bridge는 보드 안 두 칩 사이, Wi-Fi는 노트북과 보드 사이를 잇습니다.

Bridge = 한 보드 안의 두 컴퓨터가 '이름 + 값'으로 대화하게 해 주는 내선 전화

PART 2 [개념] Bridge / RPC

다른 프로세서에 등록한 함수를 이름으로 호출합니다

STM32 · C++

```
provide("set_vision_block", f)
notify("distance", 347)
```

Linux · Python

```
notify("set_vision_block", True)
provide("distance", on_distance)
```



API	역할	오늘
provide	내 함수를 등록	양쪽에서 사용
notify	상대에게 전달, 결과 대기 없음	센서값·정지 상태
call	호출 결과를 기다림	콜백 안에서 사용하지 않음

콜백 안의 동기 call()은 응답 대기가 얹혀 교착 상태를 만들 수 있습니다.

PART 2

실습 2

MPU Python 로그 확인

MCU가 읽은 값을 Bridge로 Linux쪽 Python까지 넘기는 단계입니다.
실습 1에서 막아둔 Bridge.notify(...)주석을 풀고 앱을 다시 실행해 보낸 거리값이 Bridge를 건너
Python 로그에 찍히는지 확인합니다. 손을 움직일 때 Python쪽의 로그의 숫자도 변하면
두 프로세서 사이의 통로 연결 성공!

PART 2 [실습 2] 코드 ② · Python으로 거리 전달

sketch.ino · 코드 ② 주석 해제

```
// loop()에서 주석 해제  
Bridge.notify(  
  "distance", distance_mm  
);
```

같은 이름으로 연결

flush=True
출력 버퍼를 비워 로그에 바로 표시

python/main.py

```
from arduino.app_utils import *  
  
def on_distance(distance_mm: int):  
    print(distance_mm, flush=True)  
  
Bridge.provide(  
    "distance", on_distance  
)  
App.run()
```

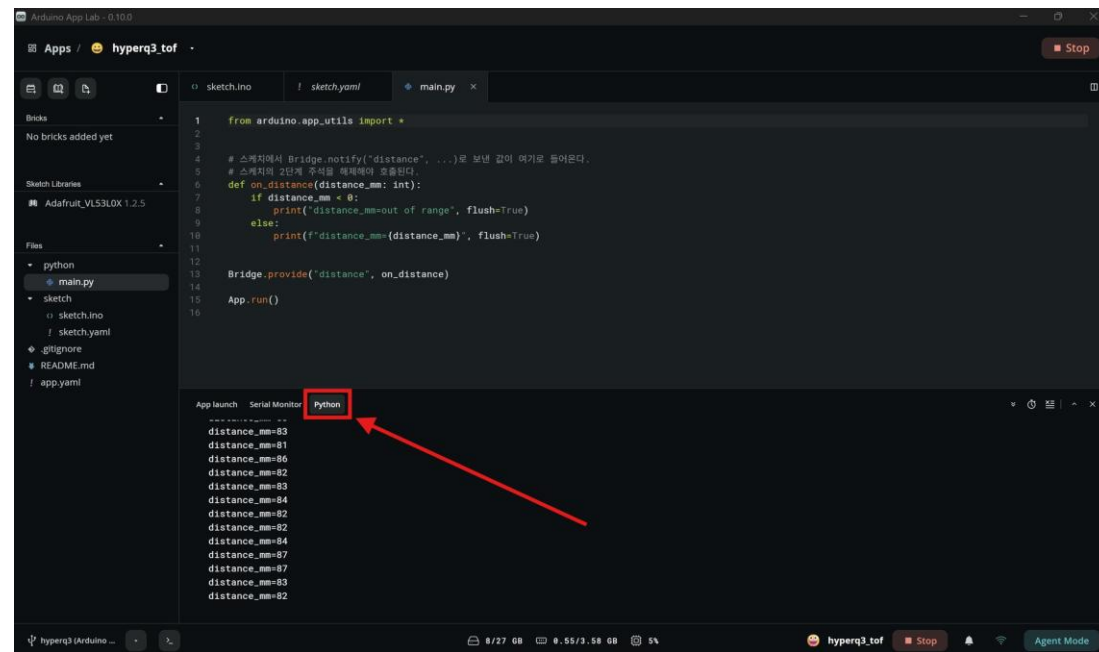
MCU 센서값 확인이 끝나면 주석을 해제하고 저장한 뒤 앱을 다시 실행합니다.

PART 2 [실습 2] MPU Python 로그 확인

코드 ② 적용 후 실행

- ① Bridge.notify(...) 주석 해제
python/main.py 코드 확인
- ② 저장 후 앱 다시 실행
- ③ Python 탭에서 거리값 확인
손을 움직여 값이 변하는지 관찰

MPU Python 로그



Python에 도착한 값으로 파일 저장, 그래프, 웹 전송 등을 할 수 있습니다. 무한한 응용 가능!

Part 3-1

Linux와 SSH 연결

[개념] 리눅스가 맡는 일 : 컴퓨터가 하는 일을 맡는다!

카메라 영상

드라이버로 장치를 열고
프레임을 계속 읽습니다.

AI 모델

모델 실행 환경에서
영상의 물체를 추론합니다.

웹서버

여러 브라우저에
화면과 결과를 전달합니다.

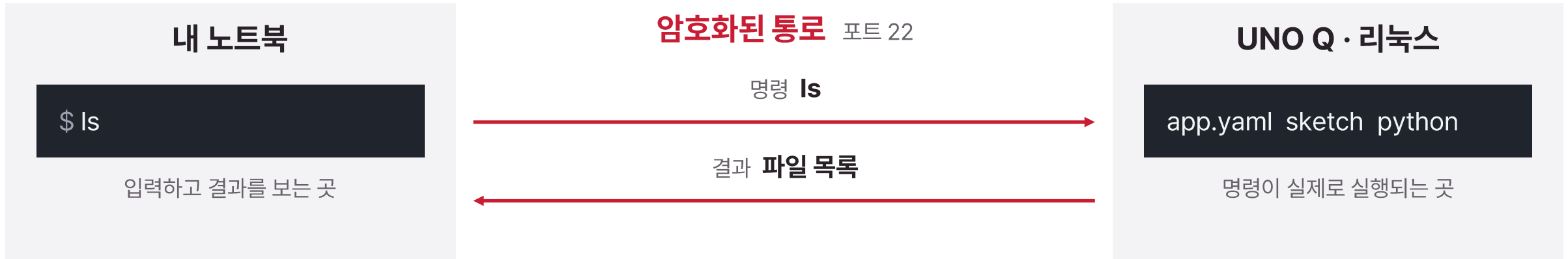
리눅스는 여러 프로그램과 장치를 함께 관리합니다.

카메라 · AI 모델 · 웹서버가 동시에 돌 수 있는 것은 운영체제 덕분입니다



[개념] SSH란? · 노트북으로 보드를 원격 조종하기

SSH(Secure Shell) = 내 노트북을 보드의 키보드·모니터처럼 쓰게 해 주는 암호화된 원격 접속



접속 명령 한 줄 읽기

ssh	arduino	@	<팀_IP>
접속 프로그램	보드의 계정	~의	보드 주소

SSH와 원격 데스크톱의 차이

SSH 글자만 주고받아 가볍고 빠름 · 여러명 동시 접속
원격 데스크톱 화면 전체를 가져옴 · 편집 담당 1명

터미널 창은 내 노트북에 있지만, 명령은 보드에서 실행됩니다 · 끝낼 때는 exit

[개념] 터미널 · 명령어 한 줄 읽는 법

터미널은 마우스 대신 글자로 컴퓨터에 일을 시키는 창입니다

① 프롬프트 읽기

arduino @ 보드이름 : ~ \$

누가
사용자 계정

어디서
접속한 컴퓨터

현재 위치
홈 폴더

입력 대기
명령 기다림

예: cd ArduinoApps 실행 후

arduino@보드이름:~/ArduinoApps\$

~ 자리에 지금 있는 폴더가 표시됩니다

② 명령어 구조

ls

명령
무엇을 할지

-l

옵션
어떻게 · - 로 시작

/dev

대상
어디에 · 무엇에

예시

cd ArduinoApps

명령 + 대상 · 폴더로 이동

ls -la

명령 + 옵션 · 숨김 파일까지 자세히

cat sketch.ino

명령 + 대상 · 파일 내용 보기

출력 없이 다음 프롬프트가 나오면 대부분 성공입니다 · 띄어쓰기와 대소문자도 구분합니다

PART 3 [개념] 명령어 기초 · 길 찾기

명령	뜻
pwd / ls -la	현재 위치 / 파일 목록
cd / cd .. / cd ~	이동 / 상위 / 홈
cat / mkdir	내용 보기 / 폴더 만들기
cp / mv	복사 / 이동·이름 변경
hostname -I	보드 IP
whoami / who	계정 / 접속자
clear / exit	화면 정리 / 접속 종료

```
/
└─ home
   └─ arduino
      └─ ArduinoApps
         └─ q-rover-02
            ├── app.yaml
            ├── sketch
            └─ python
```

절대경로 /home/arduino/ArduinoApps
상대경로 ArduinoApps

Tab 자동완성 · ↑ 이전 명령 · Ctrl+C 실행 중단 · Ctrl+D 입력 종료

실습 3-1

SSH- 내 컴퓨터로 보드 안에 들어가보기

로봇이나 보드처럼 화면·마우스 없이 쓰는 리눅스 기기는 대부분 터미널에 글자로 명령을 입력해서 다룹니다. 이번에는 SSH로 보드에 들어가 터미널에서 cd, ls, cat 세 가지만으로 지난주 앱 폴더를 찾아가고, 스케치 코드를 터미널에 띄워 봅니다.
마우스 없이 경로를 따라 이동하는 감각을 익혀봅시다!

PART 3 [실습 3-1] SSH 연결하기 · 노트북에서 보드에 명령(무선)

SSH 를 통해 노트북에서 보드로 무선으로 명령을 전달해봅시다.

암호화된 연결

내 노트북

UNO Q의 리눅스

키보드 입력과 화면 표시

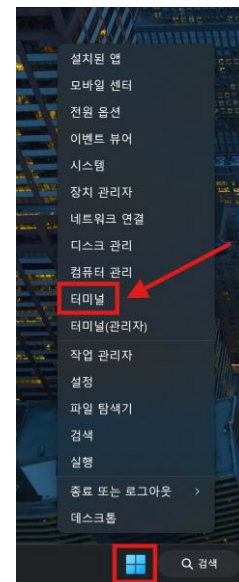
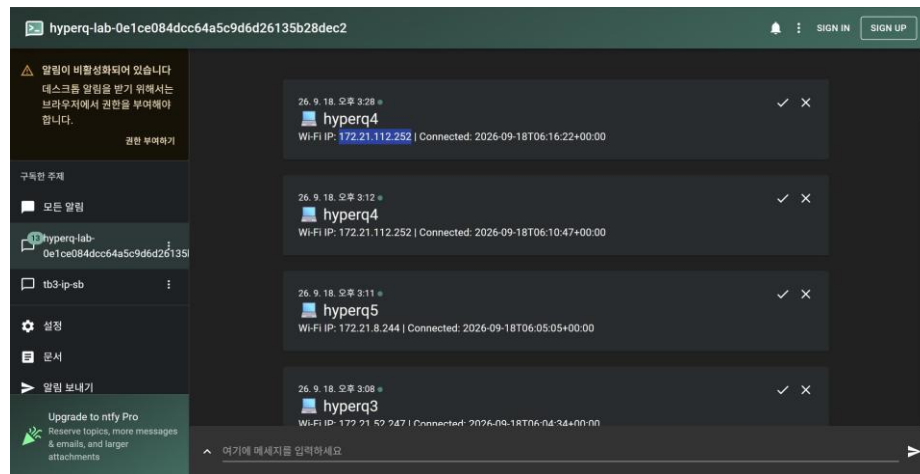
명령과 프로그램이 실제로 실행되는 곳

연결하는 법

- ① 보드와 같은 KHU Wi-Fi에 접속
SSH는 같은 네트워크 안에서만 연결됩니다
- ② 채널에서 우리 팀 보드의 최신 IP 확인
보드의 IP 주소를 알아야 접속할 수 있습니다
- ③ 시작 버튼 우클릭 → 터미널 열기

팀 IP 확인 채널 열기

<https://ntfy.sh/hyperq-lab-0e1ce084dcc64a5c9d6d26135b28dec2>



PART 3 [실습 3-1] SSH 연결하기 · 접속 화면 따라가기

① 노트북 터미널에 입력 · <팀_IP>를 실제 주소로 교체

```
ssh arduino@<팀_IP>
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\seong> ssh arduino@hyperq2
The authenticity of host 'hyperq2 (fe80::9d26:15ae:79fb:3ef4%9)' can't be established.
ED25519 key fingerprint is SHA256:MTfI+0Q1+oNyAAE0CVseKd64Fj8EnD6MdKjDvglin+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? |
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\seong> ssh arduino@hyperq2
The authenticity of host 'hyperq2 (fe80::9d26:15ae:79fb:3ef4%9)' can't be established.
ED25519 key fingerprint is SHA256:MTfI+0Q1+oNyAAE0CVseKd64Fj8EnD6MdKjDvglin+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hyperq2' (ED25519) to the list of known hosts.
arduino@hyperq2's password: |
```

② 첫 접속 확인에 yes 입력

```
arduino@hyperq2: ~
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\seong> ssh arduino@hyperq2
The authenticity of host 'hyperq2 (fe80::9d26:15ae:79fb:3ef4%9)' can't be established.
ED25519 key fingerprint is SHA256:MTfI+0Q1+oNyAAE0CVseKd64Fj8EnD6MdKjDvglin+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hyperq2' (ED25519) to the list of known hosts.
arduino@hyperq2's password:
Linux hyperq2 6.16.7-g0dd6551ae96b #1 SMP PREEMPT Wed Dec 10 14:36:39 UTC 2025 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
arduino@hyperq2:~$ |
```

③ 비밀번호 hyper123 입력

④ arduino@보드이름:~\$ 가 보이면 성공
=> 이때 터미널은 "아두이노 Q보드"의 터미널이다

비밀번호는 입력해도 표시되지 않습니다.

PART 3 [실습3-1] SSH 연결 상태에서 지난주 앱 폴더 찾기

① 홈으로 이동

```
cd ~
```

② 목록에서 ArduinoApps 확인

```
ls
```

③ 앱 폴더로 이동

```
cd ArduinoApps
```

④ 목록에서 지난주 앱 이름 확인

```
ls
```

```
arduino@hyperq2: ~/Arduino. x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\seong> ssh arduino@hyperq2
The authenticity of host 'hyperq2 (fe80::9d26:15ae:79fb:3ef4%9)' can't be established.
ED25519 key fingerprint is SHA256:MTfI+0Q1+oNyAAE0CVseKd64Fj8EnD6MdKjDvglin+A.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hyperq2' (ED25519) to the list of known hosts.
arduino@hyperq2's password:
Linux hyperq2 6.16.7-g0dd6551ae96b #1 SMP PREEMPT Wed Dec 10 14:36:39 UTC 2025 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
arduino@hyperq2:~$ ls
ArduinoApps  lost+found
arduino@hyperq2:~$ pwd
/home/arduino
arduino@hyperq2:~$ cd ArduinoApps/
arduino@hyperq2:~/ArduinoApps$ ls
hyper  q-rover-01
arduino@hyperq2:~/ArduinoApps$ cd q-rover-01/
arduino@hyperq2:~/ArduinoApps/q-rover-01$ ls
app.yaml  python  README.md  sketch
arduino@hyperq2:~/ArduinoApps/q-rover-01$ |
```

각 명령을 하나씩 실행합니다.

PART 3 [실습3-1] sketch.ino를 터미널에 출력

① 앞에서 확인한 앱 폴더로 이동

```
cd q-rover-01
```

② sketch 폴더가 있는지 확인

```
ls
```

③ 스케치 폴더로 이동

```
cd sketch
```

④ 소스 코드 출력

```
cat sketch.ino
```

```
arduino@hyperq2: ~/Arduino
hyper q-rover-01
arduino@hyperq2:~/ArduinoApps$ cd q-rover-01/
arduino@hyperq2:~/ArduinoApps/q-rover-01$ ls
app.yaml  python  README.md  sketch
arduino@hyperq2:~/ArduinoApps/q-rover-01$ cd sketch
arduino@hyperq2:~/ArduinoApps/q-rover-01/sketch$ ls
sketch.ino  sketch.yaml
arduino@hyperq2:~/ArduinoApps/q-rover-01/sketch$ cat sketch.ino
// =====
// Q-ROVER 1차시 : DRV8833으로 바퀴 굴리기
// 보드 : Arduino UNO Q (Zephyr core)
// =====

#include "Arduino_RouterBridge.h" // App Lab 콘솔(Monitor)을 쓰기 위해 필요
// 리눅스와 통신하는 Bridge는 2차시부터 씁니다

// DRV8833은 핀 4개로 모터 2개를 굴린다.
// 한 모터당 2핀(IN1/IN2)을 쓰고, 네 핀 모두에서 PWM이 나가야 한다.
// UNO Q에서 analogWrite()가 되는 핀은 3, 5, 6, 9 - 딱 네 개다.
const int AIN1 = 5; // 왼쪽 모터
const int AIN2 = 3;
const int BIN1 = 6; // 오른쪽 모터
const int BIN2 = 9;
// 방향이 안맞으면 핀 번호끼리 바꿔보기!

const int SPEED = 130; // 0 ~ 255

// -----
// 바퀴 하나 굴리기
// speed > 0 : 앞으로
// speed < 0 : 뒤로
// speed = 0 : 짐 빼기 (coast)
// -----

void wheel(int in1, int in2, int speed) {
    speed = constrain(speed, -255, 255);

    if (speed > 0) {
        analogWrite(in1, 255);
        analogWrite(in2, 255 - speed); // 반전 PWM (slow decay) - 슬라이드 12 참고
    } else if (speed < 0) {
        analogWrite(in2, 255);
        analogWrite(in1, 255 + speed);
    } else {

```

앱 이름이 다르면 ①을 바꿉니다. cat은 내용을 보여주며 파일을 수정하지 않습니다.

Part 3-2

Linux로 보드 수사하기
보드 안에 무슨 일이 일어나고 있을까?

PART 3 [개념] 리눅스에서는 장치도 파일 - 연결 장치 확인하기

① 입력할 명령: 연결된 장치 확인

```
ls -l /dev/video*
```

맨 앞 글자로 구분
c 문자 장치
- 일반 파일
d 디렉터리

출력 예시

```
crw-rw---- ... /dev/video0  
crw-rw---- ... /dev/video1
```

USB 카메라



커널 / V4L2



/dev/videoN



OpenCV

드라이버에서 하드웨어를 제어합니다. 프로그램은 장치 파일을 열어 영상을 읽습니다.

한 카메라가 여러 video 노드를 만들 수 있습니다.

PART 3 [개념] 장치와 시스템 정보 읽기

① USB 장치가 연결된 기록 확인

```
dmesg | tail -20
```

② 카메라 이름과 장치 경로 확인

```
v4l2-ctl --list-devices
```

③ CPU 정보 앞 20줄 읽기

```
cat /proc/cpuinfo | head -20
```

④ 메모리 정보 앞 5줄 읽기

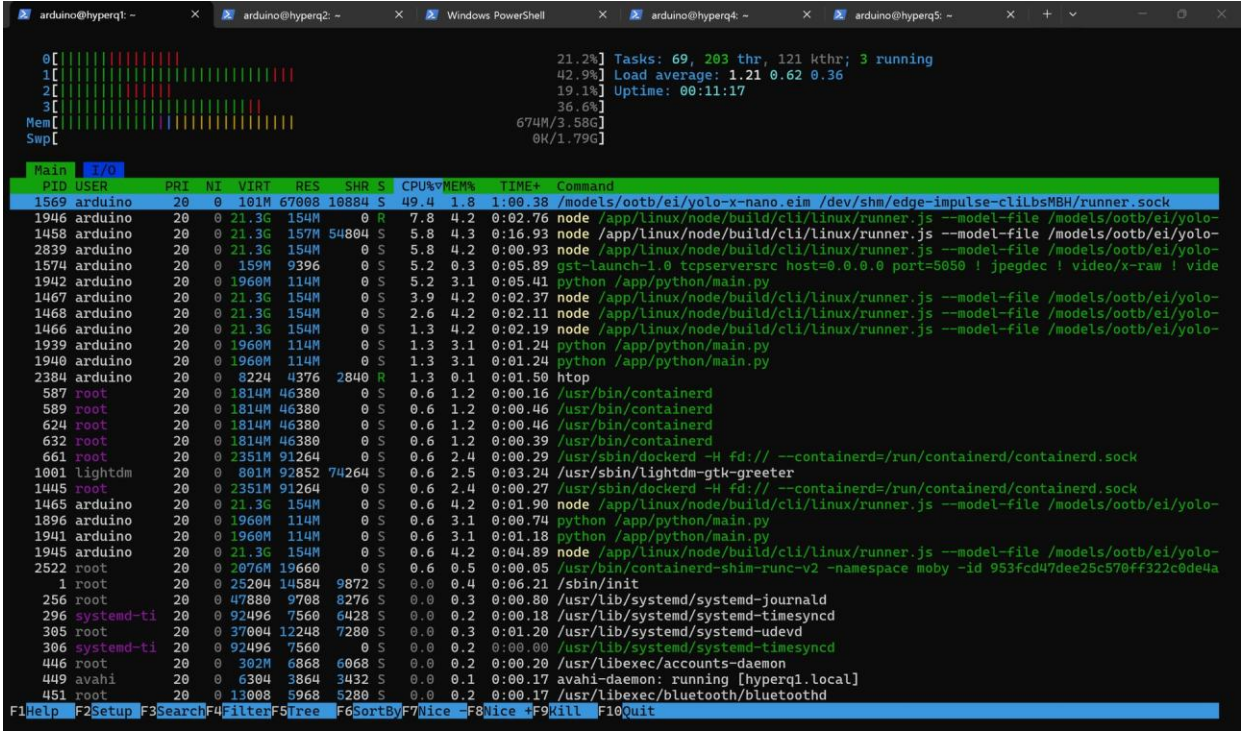
```
cat /proc/meminfo | head -5
```

/proc 아래 파일은 커널이 제공하는 시스템 정보입니다.

PART 3

[개념] 프로세스 · 보드에서 도는 프로그램

명령	확인 대상
ps aux	프로세스 목록
htop / top	CPU·메모리 실시간 부하
docker ps	실행 중인 컨테이너
ss -tlnp	열린 TCP 포트
free -h / df -h	메모리 / 디스크 여유



htop 실행 예시

```
ss -tlnp | grep -E '7000|4912|5050'
```

PART 3 [개념] 파이프와 리다이렉션

ls /dev



grep video



wc -l

기호	기능
	출력을 다음 명령의 입력으로
>	파일에 덮어쓰기
>>	파일에 이어쓰기
head / tail	앞 / 뒤 몇 줄 보기

① 카메라 장치만 출력

```
ls /dev | grep video
```

② IP를 파일에 저장

```
hostname -l > ~/myip.txt
```

③ 저장 결과 읽기

```
cat ~/myip.txt
```

[개념] 프로그램 · 프로세스 · 컨테이너

htop과 docker ps가 보여 주는 것이 무엇인지 구분해 봅시다

프로그램

비유 · 레시피

디스크에 저장된 코드 파일. 저장만 되어 있을 때는 아무 일도 하지 않습니다.

예: main.py · sketch.ino

실행

프로세스

비유 · 요리 중인 요리사

실행 중인 프로그램. CPU와 메모리를 쓰며 일하고, 번호(PID)로 구분합니다.

확인: ps aux · htop

환경째 포장

컨테이너

비유 · 밀키트

프로그램과 필요한 라이브러리·설정을 한 상자에 담아 어디서나 똑같이 실행합니다.

확인: docker ps

CPU 요리사의 손 · 일을 처리하는 능력 (htop의 CPU%)

메모리 조리대 · 작업 중인 데이터를 올려 두는 공간 (free -h)

오늘 AI 모델은 컨테이너 안에서 돌고, Python 프로그램과 포트로 데이터를 주고받습니다

실습 3-2

5인이 동시에 보드 수사해보기

실행 담당 1명이 보드에 Object Detection 예제를 켜두고, 나머지 팀원이 같은 보드에 SSH로 접속해 각자 다른 각도로 "지금 보드에서 무슨 일이 벌어지고 있나"를 확인합니다.

각자 결과를 ~/practice/내이름 폴더에 기록하고, 다섯 조각을 합쳐 "카메라 → Python → 모델 → 브라우저" 경로를 팀이 직접 설명하는 게 최종 목표예요. 보드 한 대를 여럿이 쓰기 때문에 앱 실행·수정은 한 번에 한 명만 하고, 편집 담당 1명은 원격 데스크톱으로 들어갑니다.

PART 3 보드 1대를 함께 쓰는 규칙

작업	담당
읽기·조사	전원 자유
파일 쓰기	자기 practice 폴더
앱 실행·중지·수정	한 번에 1명
브라우저 결과 확인	전원 가능

SSH · 5명

명령 실행, 상태 조사, 로그 확인

원격 데스크톱 · 1명

보드의 바탕화면을 보며
App Lab에서 코드 편집

편집 담당이 저장한 뒤 원격 화면을 닫고, SSH에서 빌드·실행합니다.

원격 편집이 느리면 지난주처럼 유선 App Lab에서 편집·업로드해도 됩니다.

PART 3 실습 3 준비 · 같은 보드에 접속

① 전원 · 접속자 확인

```
who
```

② 전원 · 터미널 파일 확인

```
ls /dev/pts
```

③ 같은 계정의 터미널에 메시지 전송

```
for t in /dev/pts/[0-9]*; do  
    printf "1팀 접속 완료\n" > "$t"  
done
```

③은 for부터 done까지 한 덩어리로 복사해서 실행해 주세요.

④ 실행 담당 1명만 · 객체인식 예제 시작

```
arduino-app-cli app start examples:inspirational/video-generic-object-detection
```

/dev/pts의 숫자는 터미널 번호입니다. 한 사람이 여러 터미널을 열 수도 있습니다.

PART 3 [실습 3-2] · 5인 동시 수사

한 노트북에서 터미널 여러 개를 열어서 해도 됩니다.

① 부하 담당

htop

CPU를 많이 쓰는
프로세스는?

② 장치 담당

ls -l /dev/video*

영상 입력 노드와
카메라 이름은?

③ 컨테이너 담당

docker ps

컨테이너 이름과
이미지 이름은?

④ 포트 담당

ss -tlnp

5050 / 4912 / 7000
각 포트 역할은?

⑤ 로그 담당

app logs ... --follow

어떤 라벨이
반복해서 보이냐?

각자 맡은 역할의 명령만 실행합니다.

다섯 결과를 합쳐 카메라 → Python → 모델 → 브라우저의 경로를 설명합니다.

PART 3 [실습 3-2] · 로그 확인과 조사 기록

로그 담당 · 실행 후 출력 관찰, Ctrl+C로 보기 종료

```
arduino-app-cli app logs examples:inspirational/video-generic-object-detection --follow
```

전원 · 내이름을 바꿔 각자 기록 폴더 생성

```
mkdir -p ~/practice/내이름
```

다음 · 만든 폴더로 이동

```
cd ~/practice/내이름
```

다음 · 조사 결과 저장

```
docker ps > mynote.txt
```

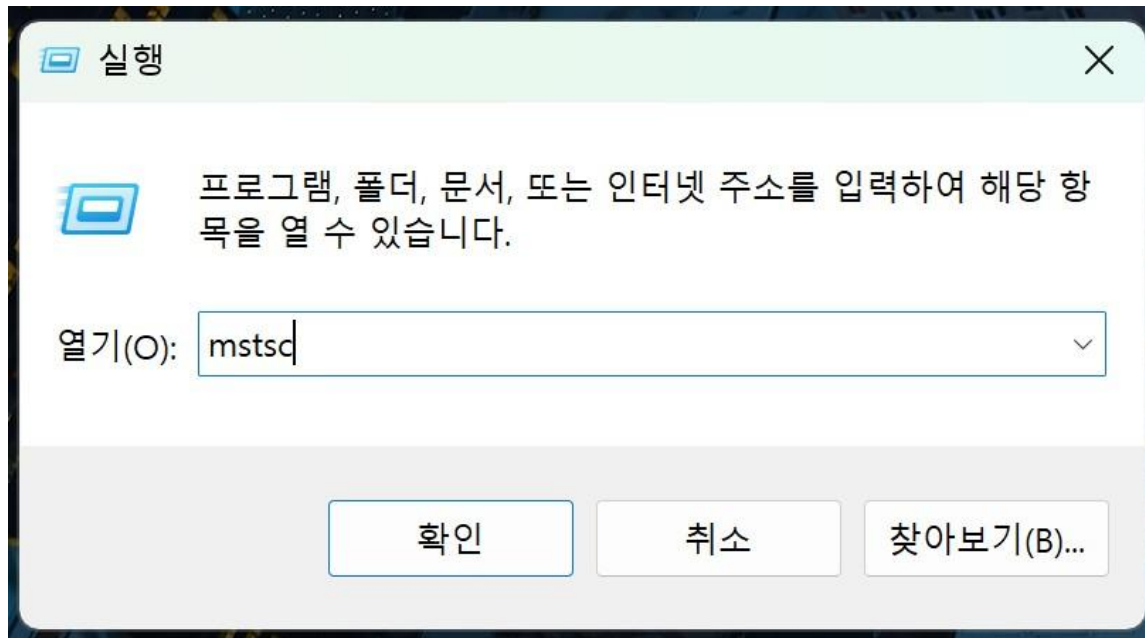
다음 · 저장 결과 읽기

```
cat mynote.txt
```

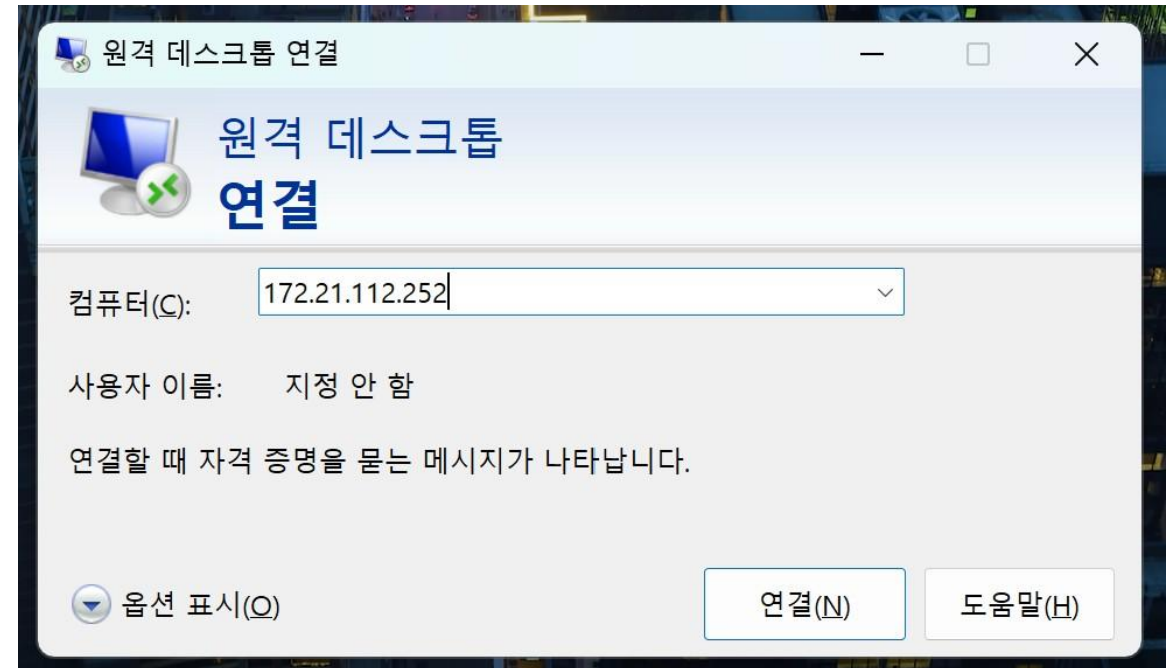
같은 arduino 계정이므로 홈 폴더를 공유합니다. 자기 폴더에서 기록합니다.
부하·장치·컨테이너·포트·로그 결과를 모아 데이터 경로를 설명합니다.

PART 3 [추가 실습] 원격 데스크톱 : 실제 보드의 컴퓨터 GUI를 확인해보자

팀원 중 한명만 보드의 바탕화면에 접속합니다. XRDP는 미리 설정돼 있습니다.

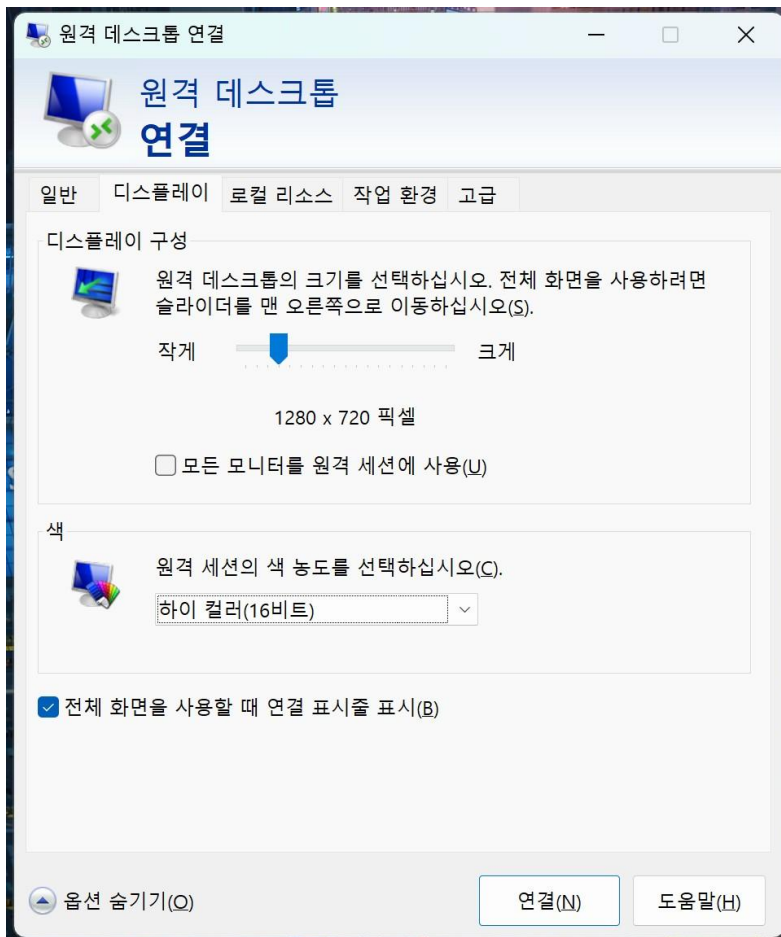


1 Win+R을 누르고 mstsc 입력



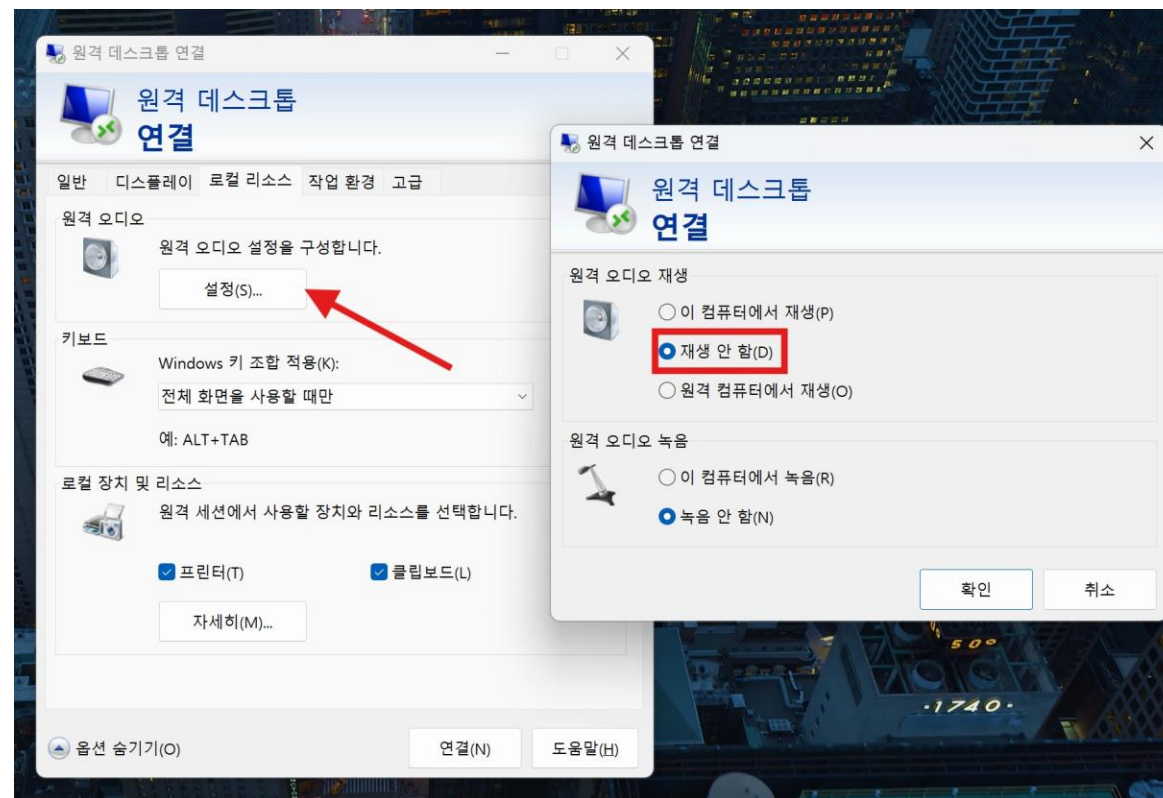
2 내 팀 IP 입력 후 옵션 표시

[추가 실습] 원격 데스크톱 : 실제 보드의 컴퓨터 GUI를 확인해보자

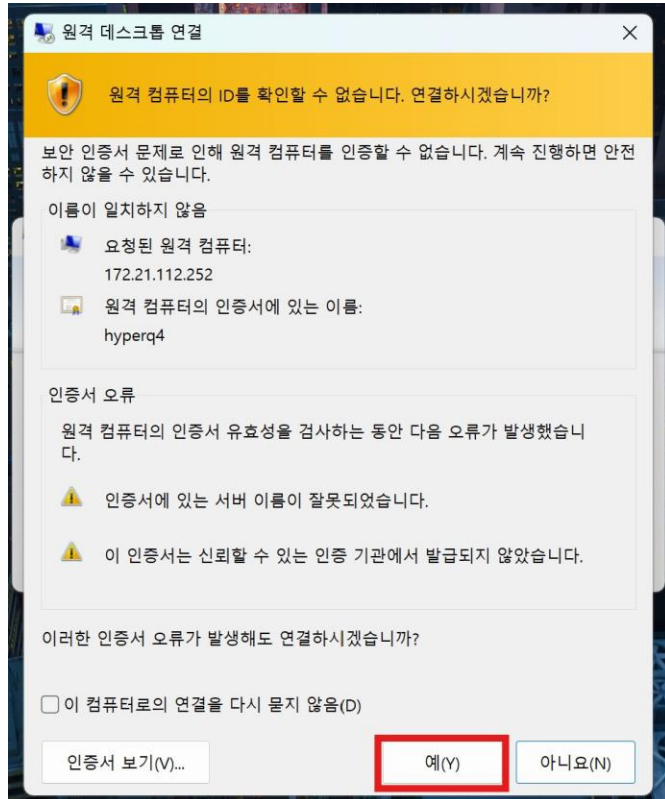


3 디스플레이 크기 설정

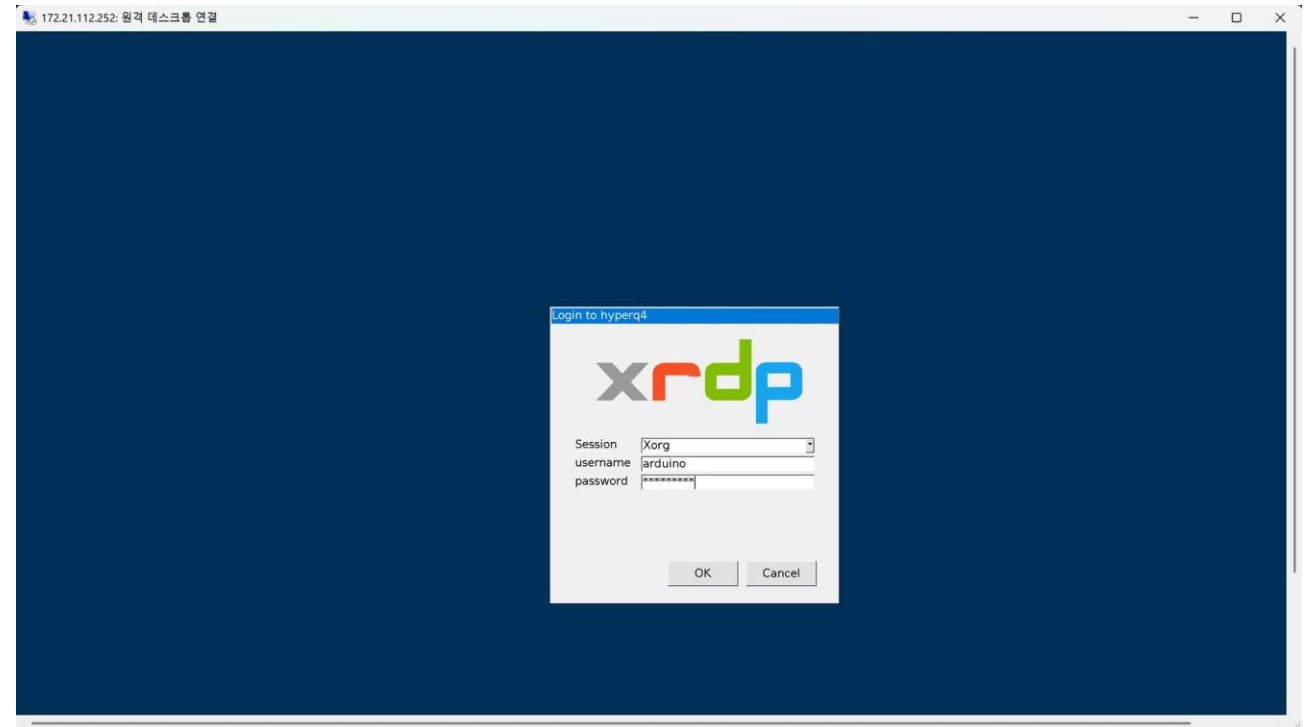
화면이 너무 작으면 재접속해서 키워주세요

4 로컬 리소스 → 오디오 설정
원격 오디오 재생: 재생 안 함

PART 3 [추가 실습] 원격 데스크톱 : 실제 보드의 컴퓨터 GUI를 확인해보자



5 대상 IP를 확인하고 연결 진행



6 XRDP 로그인

사용자 arduino / 비밀번호 hyper123

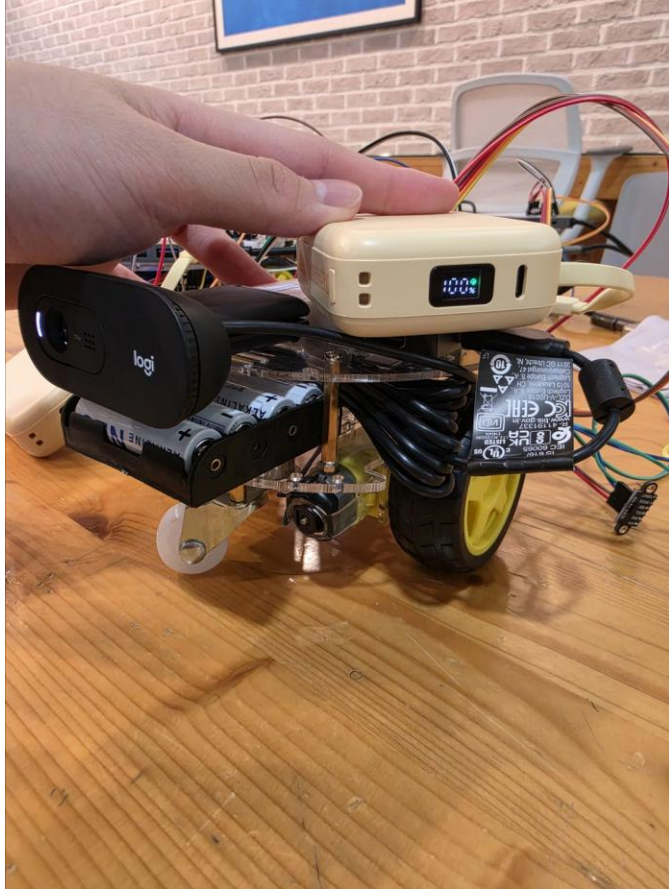
=> 이후 보이는 화면은 실제 Arduino Q에 들어있는 컴퓨터의 화면입니다.

PART 1

Part 4

카메라를 연결해 YOLO/OpenCV로 객체 인식하기

PART 4 카메라와 전원 연결

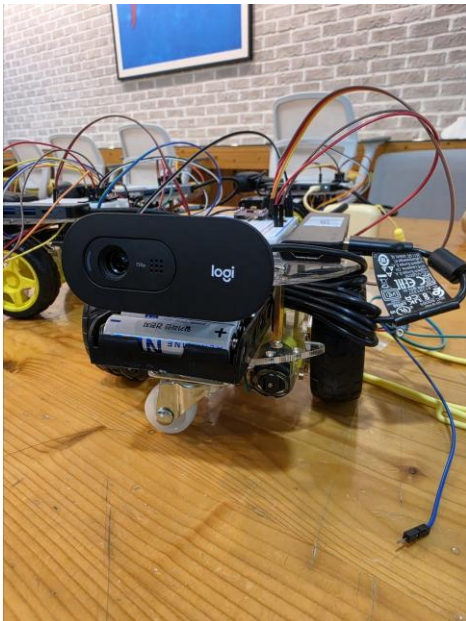


준비물

Logitech C270 USB 웹캠
외부 전원 입력 USB-C 허브
허브용 5 V / 3 A 전원
같은 Wi-Fi의 노트북

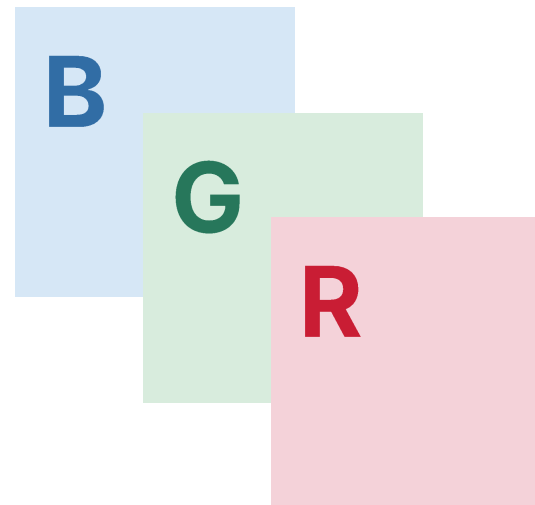
웹캠을 먼저 꽂고 허브에 전원을 공급한 뒤
마지막에 UNO Q를 연결합니다.
인식되지 않으면 연결·전원 상태부터 확인합니다.

PART 4 [개념] 이미지는 숫자 배열



191	210	190	185	185	157
19	100	213	79	25	146
32	35	33	37	100	255
15	12	33	21	101	86
64	54	33	13	40	98
89	88	78	74	66	95

축소한 이미지의 B 채널 값



OpenCV 기본 순서: B, G, R

$640 \times 480 \times 3 \times 1 \text{ byte} = 921,600 \text{ bytes}$

한 프레임 = 900 KiB $\approx$ 0.922 MB 10 fps $\approx$ 9.22 MB/s

MCU 전체 SRAM 786 KB보다 큽니다. 영상은 MPU가 처리합니다.

[개념] OpenCV · 영상 입출력과 전처리



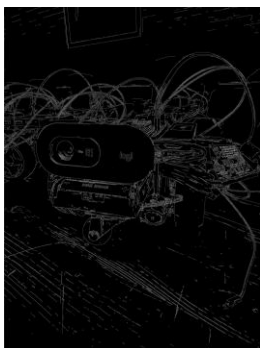
원본



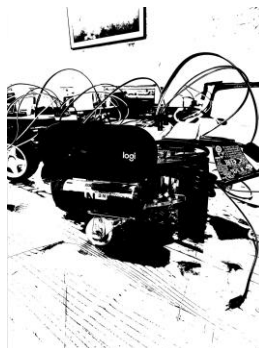
그레이스케일



가우시안 블러



Canny 엣지



이진화



리사이즈

이 앱에서는 OpenCV가 프레임을 준비하고, YOLOX 모델이 물체를 판단합니다.

[개념] AI 기초 · 학습과 추론

AI 모델은 먼저 공부(학습)를 마치고, 그다음 시험(추론)을 봅니다

학습

Training

오늘은 하지 않음

정답이 달린 사진 수
십만 장

COCO 데이터셋

맞힐 때까지 조금씩
고치기

수많은 반복

모델 파일

yolo-x-nano.eim

학습 결과를 가져다 씀

추론

Inference

오늘 하는 일

새 카메라 사진

640 × 480

학습된 모델로 계산

YOLOX-Nano

bottle · 0.87

+ 박스 위치

데이터셋

공부용 사진과 정답의 모음

라벨

사진 속 물체의 정답 이름 (bottle, cup ...)

모델

학습 결과가 담긴 파일

신뢰도

모델이 매긴 자신감 점수 · 정답 확률은 아님

새 물체를 알아보게 하려면 그 물체 사진으로 다시 학습해야 합니다 — 다음 시간 주제

PART 4 [개념] 분류 · 검출 · 분할

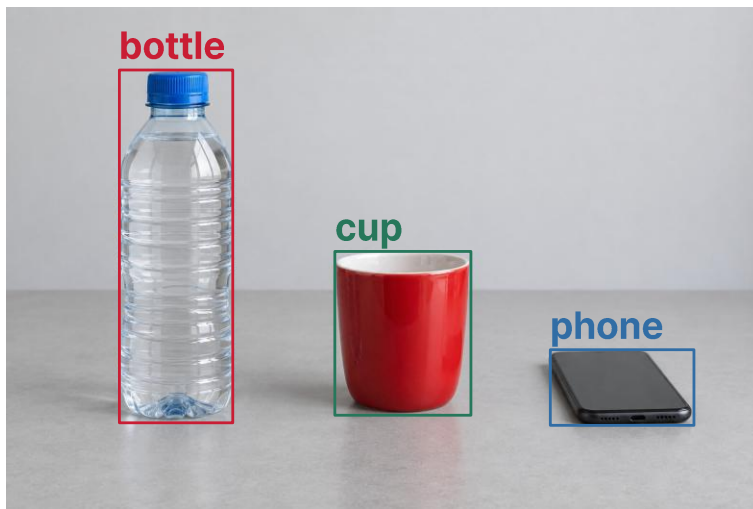
분류 Classification



대표 라벨

이미지에 무엇이 있는가

검출 Detection



라벨 + 경계 상자

무엇이 어디에 있는가

분할 Segmentation



픽셀별 마스크

정확히 어느 픽셀인가

오늘 사용하는 것은 검출입니다. 박스와 라벨을 로버의 판단에 연결합니다.

박스와 마스크는 개념 예시

[개념] YOLO · You Only Look Once

2-stage

입력 이미지



후보 영역



분류 + 박스 보정

1-stage

입력 이미지



한 번의 신경망 추론



박스 + 클래스

```
label    : "bottle"  
confidence : 0.87  
x, y, w, h : 120, 80, 60, 180
```

신뢰도: 모델의 점수입니다.

=> 정답일 확률로 단정하지 않는다!

속도와 정확도는 모델·하드웨어에 따라 달라집니다.

PART 4 [개념] 보드의 모델 · YOLOX-Nano

항목	수업 예제
모델	YOLOX-Nano
실행 파일	yolo-x-nano.eim
형식	Edge Impulse 실행 모델
클래스	COCO 80종
오늘의 작업	학습이 끝난 모델로 추론

COCO 라벨 예시

person cup cell phone
clock book chair
laptop mouse keyboard
scissors potted plant
cat dog

오늘 사용 예정 라벨: bottle

새로운 라벨을 인식시키려면 그 라벨을 학습한 모델이 필요합니다.

PART 4 [개념] 이 앱에서 OpenCV가 하는 일

```
# 브릭 코드의 핵심 발췌
cap = cv2.VideoCapture(
    path, cv2.CAP_V4L2
)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cap.set(cv2.CAP_PROP_FPS, 10)
ret, frame = cap.read()
ok, jpeg = cv2.imencode(".jpg", frame)
```

장치 파일을 프레임으로

- 1 V4L2로 카메라 열기
- 2 작은 버퍼로 지연 줄이기
- 3 크기·FPS 요청 후 실제 값 확인
- 4 배열 한 장 읽기
- 5 JPEG로 압축해 전송

JPEG 크기는 장면과 품질에 따라 달라집니다.
압축은 전송량을 줄이기 위한 선택입니다.

원본 v4l_camera.py 열기

https://github.com/arduino/app-bricks-py/blob/main/src/arduino/app_peripherals/camera/v4l_camera.py

실습 4

객체 인식 예제 실행해보기

예제 코드를 실행하고 브라우저로 <팀_IP>:7000에 들어가 컵, 휴대폰, 생수병에 박스와 라벨이 뜨는지 확인하고, 신뢰도 임계값을 0.0 / 0.5 / 1.0으로 바꾸며 오검출과 놓침이 어떻게 달라지는지 비교합니다. 이번 실습에선 app lab의 미리 학습된 yolo모델을 가져오는 것이기 때문에 직접 모델을 학습하진 않고 결과 확인을 하며 동작 원리를 이해합니다. 다음 시간부터는 yolo모델을 직접 학습해봅니다.

PART 5 [실습 4] 객체인식 예제 실행

① 팀원 1명만 실행

```
arduino-app-cli app start examples:inspirational/video-generic-object-detection
```

② 로그 담당 터미널에서 실행

```
arduino-app-cli app logs examples:inspirational/video-generic-object-detection --follow
```

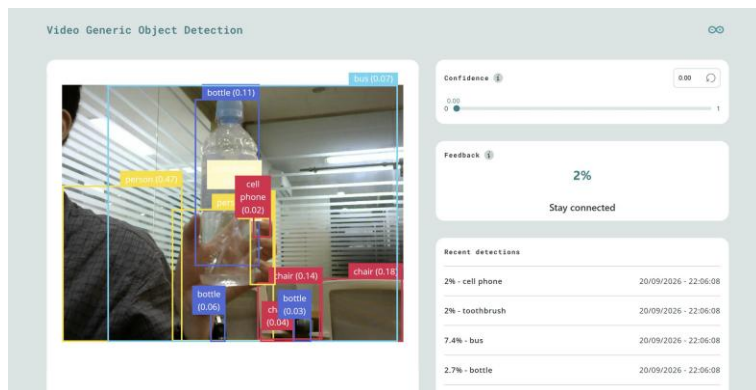
③ 실습이 끝났을 때만 중지

```
arduino-app-cli app stop examples:inspirational/video-generic-object-detection
```

브라우저의 주소창에 다음을 입력합니다: <팀_IP>:7000
컵·휴대폰·생수병의 라벨과 박스 확인
Ctrl+C는 로그 보기만 끝냅니다.

PART 5 [실습 4] Confidence Threshold

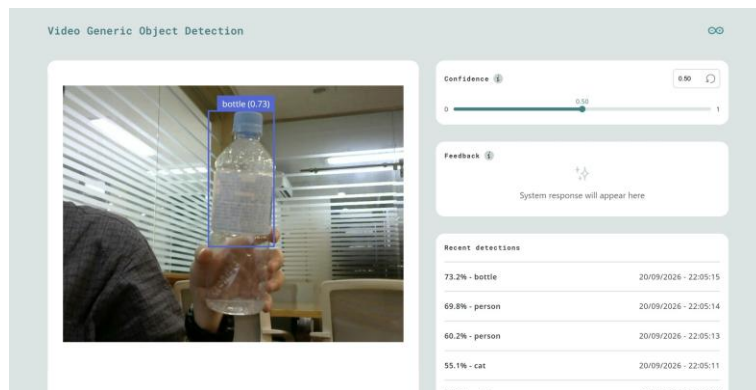
임계값을 직접 올리고 내리며 카메라를 확인해봅니다.



임계값 0.0

낮추면

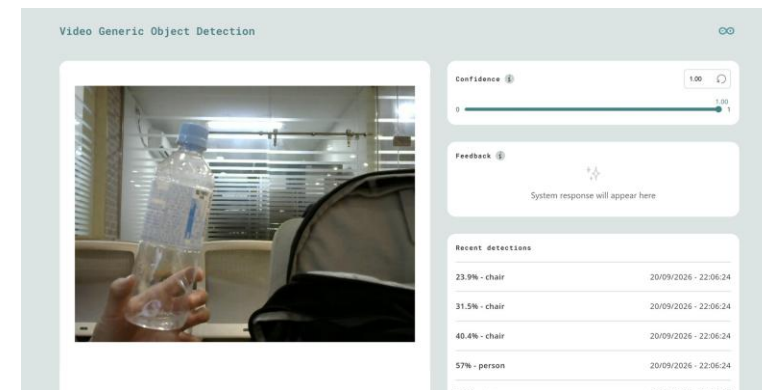
검출을 더 많이 통과시킴
오검출이 늘어날 수 있음



임계값 0.5

높이면

검출을 더 엄격하게 통과시킴
실제 병도 놓칠 수 있음



임계값 1.0

Part 5

객체 인식 예제 코드 뜯어보기

```
from arduino.app_utils import App
from arduino.app_bricks.web_ui import WebUI
from arduino.app_bricks.video_objectdetection \
    import VideoObjectDetection
```

```
ui = WebUI()
detector = VideoObjectDetection(
    confidence=0.5, debounce_sec=0.0
)
# UI 전송 콜백*을 연결
detector.on_detect_all(send_to_ui)
App.run()
```

```
bricks:
- arduino:video_object_detection: {}
- arduino:web_ui: {}
```

콜백* : 검출 결과가 도착하면 자동으로 호출하는 함수

컨테이너** : 모델과 필요한 실행 환경을 묶어 실행하는 공간

파이썬 클래스

카메라 읽기
결과 수신과 필터링
등록된 콜백* 실행

모델 컨테이너**

학습된 모델 실행
포트 5050과 4912 제공

PART 5 [개념] 전체 경로 · 포트 3개



PART 5 [개념] 두 루프와 콜백 데이터

보내는 루프

```
frame = camera.capture()
jpeg = compress_to_jpeg(frame)
tcp_socket.sendall(jpeg.tobytes())
```

받는 루프

```
message = ws.recv()
_process_message(ws, message)
# 임계값을 통과하면 콜백 실행
```

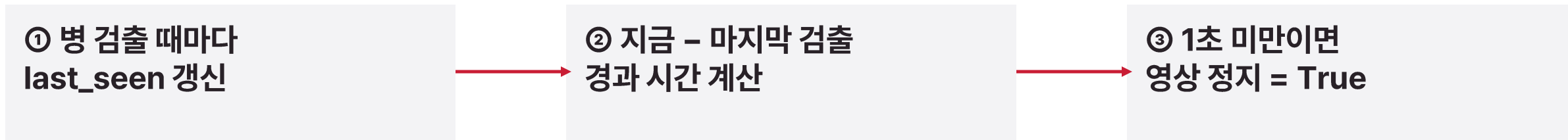
on_detect_all(cb)의 입력

```
{
  "bottle": [
    {"confidence": 0.87,
     "bounding_box_xyxy":
       (120, 80, 180, 260)}
  ],
  "person": [...]
}
```

on_detect("bottle", cb)는 개별 dict
debounce_sec는 같은 라벨의 호출 간격
콜백에서는 짧은 작업만 수행

PART 5 [개념] 검출 신호가 잠깐 끊겨도 정지 유지

병이 보일 때마다 타이머를 다시 시작합니다. 마지막 검출 뒤 1초간 정지를 유지합니다.



예: HOLD_SEC = 1.0 초



```
blocked = (time.time() - last_seen) < HOLD_SEC
```

재검출되면 1초를 다시 셉니다. 재출발은 거리 정지 조건까지 해제돼야 가능합니다.

PART 5 [실습 5] · 브릭 코드 열어보기

SSH에서 설치 경로 확인

```
python3 -c "import arduino.app_bricks.video_objectdetection as m; print(m.__file__)"
```

```
grep -n "def " <확인한_파일_경로>  
grep -n -E "5050|4912" <확인한_파일_경로>
```

찾아볼 것

camera_loop / object_detection_loop
_process_message / on_detect
confidence / debounce_sec

GitHub에서도 같은 구조를 볼 수 있습니다.

VideoObjectDetection 원본 열기
https://github.com/arduino/app-bricks-py/blob/main/src/arduino/app_bricks/video_objectdetection/__init__.py

Part6: 통합 실습

이번 통합 실습에서는 앞에서 배운 거리센서, 카메라 객체인식, Bridge 통신, 모터 제어를 하나로 연결하여 로버의 자율 정지 기능을 구현합니다. 로버는 기본적으로 직진하다가 거리센서가 20 cm 이내의 장애물을 감지하거나 카메라에서 bottle이 검출되면 정지합니다. 이후 장애물이 제거되고 bottle도 더 이상 보이지 않으면 다시 주행을 시작합니다. 카메라에서 판단한 객체 검출 결과는 MPU에서 Bridge를 통해 MCU로 전달하며, MCU가 거리센서의 결과와 함께 두 조건을 종합하여 최종적으로 모터의 주행과 정지를 결정합니다.

PART 6 통합 미션 · 코드의 빈칸 4개를 채우고 실행해봅시다

01

직진

02

20 cm 안 장애물

03

치우면 재출발

04

카메라에 bottle

05

치우면 재출발

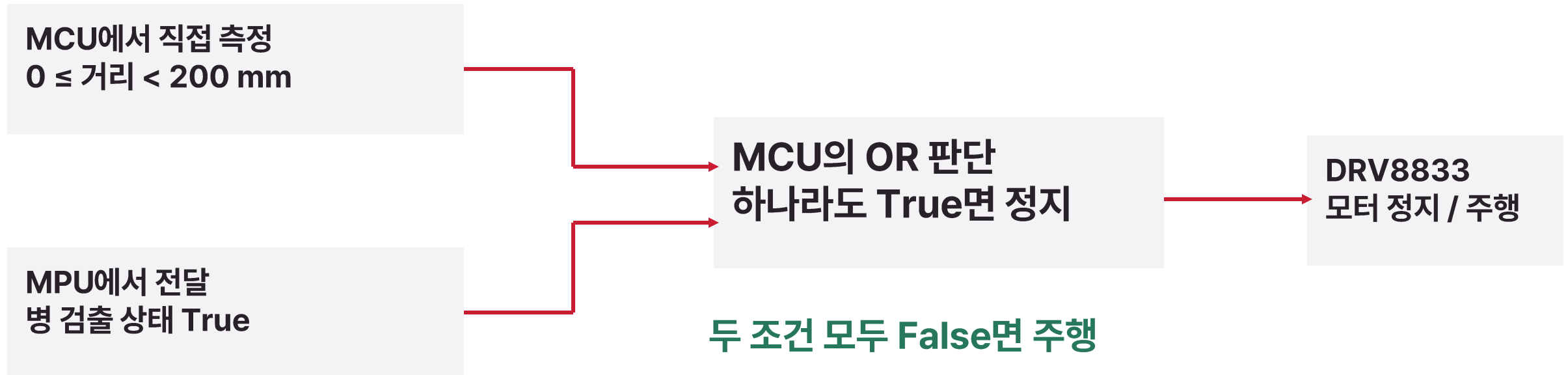
거리 정지

영상 정지

조건	판단 주체	최종 동작
거리 200 mm 미만	MCU	정지
화면에 bottle 검출	MPU	정지
두 정지 조건 모두 해제	MCU	재출발

PART 6 이번 실습의 구조 · MCU가 최종 판단

MPU는 병 검출 상태를 보내고, MCU는 거리 조건과 합쳐 모터를 제어합니다.



Bridge / RPC

이 구조를 사용합니다. 거리센서의 정지 판단은 MCU가 직접 수행합니다.

Bridge는 보드 내부 통신이며, Wi-Fi는 노트북 접속에 사용합니다.

조건과 논리 · 참/거짓으로 판단하기

코드 ③의 빈칸은 '멈춰야 할 때만 참(true)'이 되는 조건식을 고르는 문제입니다

① 비교 연산 → 결과는 참 / 거짓

기호	뜻	예 · distance_mm = 150
<	작다	150 < 200 → 참
>=	크거나 같다	150 >= 0 → 참
==	같다	150 == 200 → 거짓
!=	다르다	150 != -1 → 참

표기 주의 C++는 true / false, Python은 True / False
'같다'는 = 한 개가 아니라 == 두 개 (= 는 값 넣기)

② 논리 연산 → 조건 여러 개 합치기

&& 그리고

둘 다 참일 때만 참

|| 또는

하나라도 참이면 참

! 아니다

참 ↔ 거짓 뒤집기

A · 거리 막힘	B · 병 보임	A && B	A B
거짓	거짓	거짓	거짓
참	거짓	거짓	참
거짓	참	거짓	참
참	참	참	참

멈춰야 하는 상황을 먼저 떠올리고, 그 상황에서만 참이 되는 식을 고릅니다

PART 6 합격 조건과 흔한 실패

합격 조건

직진 / 20 cm 안에서 정지 / 병만 보여도 정지 / 치우면 재출발

증상	확인할 것
아예 안 움직임	-1을 가까운 거리로 처리했는가
병을 치워도 오래 정지	HOLD_SEC와 False 전환 로그
병을 봐도 정지하지 않음	bottle 철자, 임계값, HOLD_SEC=0 여부
덜컹거리며 전진	HOLD_SEC가 너무 짧은가
제어가 버벅임	매번 brake()를 호출하는가
Python만 반응	Bridge.provide / notify의 이름 일치

카메라 테스트에서는 거리센서를 가리지 않고, 거리값이 200 mm 이상인지 확인합니다.

Part6: 코드 확인

PART 6 코드 ③ · sketch.ino / TODO를 채워 봅시다!

```
// 모터 함수는 1차시 코드 재사용
const int STOP_MM = 200;
volatile bool visionBlocked = false;

void set_vision_block(bool blocked) {
    visionBlocked = blocked;
}

// setup()
Bridge.provide(
    "set_vision_block", set_vision_block
);
```

```
// loop() 핵심 발췌
int distance_mm =
    (m.RangeStatus != 4) ?
    m.RangeMilliMeter : -1;

// TODO ①
bool tofBlocked = ____;

// TODO ②
bool blocked = ____;

// 상태 전환 때만 brake/startDrive
```

wasMoving으로 상태 전환을 감지합니다. delay(50) 외에 측정·제동 시간도 듭니다.

PART 6 코드 ③ · sketch.ino / TODO를 골라 봅시다!

TODO ① 거리 조건

- A distance_mm < STOP_MM
- B (distance_mm >= 0) &&
(distance_mm < STOP_MM)
- C (distance_mm > 0) ||
(distance_mm < STOP_MM)
- D distance_mm != -1

TODO ② 최종 정지 조건

- A tofBlocked && visionBlocked
- B tofBlocked || visionBlocked
- C visionBlocked
- D !tofBlocked || !visionBlocked

각 보기에서 로버가 어떻게 움직일지 먼저 예상합니다.

PART 6 코드 ③ · sketch.ino / TODO 정답

TODO ① 정답 B

```
(distance_mm >= 0) &&  
(distance_mm < STOP_MM)
```

- A -1도 가까운 것으로 처리
- B 유효한 값 + 정지 거리 미만
- C 사실상 항상 정지
- D 멀리 측정돼도 정지

TODO ② 정답 B

```
tofBlocked || visionBlocked
```

- A 둘 다 막혀야 정지
- B 하나라도 막히면 정지
- C 거리센서 무시
- D 평소에도 정지

수업 예제는 -1을 장애물로 취급하지 않습니다. 측정 실패 처리와 안전성은 별도 문제입니다.

PART 6 코드 ④ · main.py / TODO를 채워 봅시다!

```
WATCH_LABEL = "???" # TODO ③
HOLD_SEC = 0.0      # TODO ④
last_seen = 0.0

def on_target(detection):
    global last_seen
    last_seen = time.time()

detector.on_detect(
    WATCH_LABEL, on_target
)
```

```
def loop():
    global vision_blocked
    blocked = (
        time.time() - last_seen
    ) < HOLD_SEC
    if blocked != vision_blocked:
        vision_blocked = blocked
        Bridge.notify(
            "set_vision_block", blocked
        )
    time.sleep(0.1)
App.run(user_loop=loop)
```

app.yaml에는 video_object_detection과 web_ui 브릭을 함께 등록합니다.

PART 6 코드 ④ · main.py / TODO를 골라 봅시다!

TODO ③ WATCH_LABEL

- A "Bottle"
- B "bottle"
- C "water bottle"
- D "물병"

TODO ④ HOLD_SEC

- A 0.0
- B 0.05
- C 1.0
- D 10.0

라벨은 실제 출력 이름과 비교하고, 시간은 검출 간격을 생각합니다.

PART 6 코드 ④ · main.py / TODO 정답

TODO ③ 정답 B

```
WATCH_LABEL = "bottle"
```

- A 대소문자가 다름
- B COCO 라벨과 정확히 일치
- C 목록에 없는 이름
- D 영문 라벨과 불일치

TODO ④ C에서 시작해 조정

```
HOLD_SEC = 1.0
```

- A 영상 정지가 동작하지 않음
- B 잠깐 누락돼도 재출발
- C 0.5~2.0 s 범위에서 실험
- D 치운 뒤에도 10초 유지

HOLD_SEC를 정한 이유를 README에 기록합니다. 변경된 상태만 MCU에 알립니다.

PART 6 Arduino App CLI · 실행과 중지

앱 ID·상태 확인

```
arduino-app-cli app list
```

스케치를 수정했을 때 · 빌드, MCU 업로드, Python 실행

```
arduino-app-cli app start user:q-rover-02 -v
```

Python 변경 후 앱 재시작

```
arduino-app-cli app restart user:q-rover-02
```

앱 중지

```
arduino-app-cli app stop user:q-rover-02
```

user:q-rover-02는 자기 앱 ID로 바꾸면 됩니다.

PART 6 Arduino App CLI · 로그와 문제 해결

Python 로그 · 계속 보기, Ctrl+C로 보기 종료

```
arduino-app-cli app logs user:q-rover-02 --follow
```

MCU 출력 · Monitor.println() 확인

```
arduino-app-cli monitor
```

최근 Python 로그 50줄 확인

```
arduino-app-cli app logs user:q-rover-02 --tail 50
```

빌드 캐시 문제일 때 · 앱을 중지한 뒤 실행하고 다시 start

```
arduino-app-cli app clean-cache user:q-rover-02
```

로그와 모니터는 서로 다른 터미널에서 엽니다.

PART 6 오늘의 제출물

```
week2/  
├ sketch/sketch.ino  
├ sketch/sketch.yaml  
├ python/main.py  
├ app.yaml  
├ README.md  
└ demo.mp4
```

20초 시연 영상
거리센서 정지와 영상 정지 모두 촬영
조교 확인 후 저장소에 push

코드와 설정

빈칸 4개 완료
브릭과 라이브러리 목록 포함

README.md

팀원 이름
HOLD_SEC 값과 선택한 이유

영상 테스트에서는 거리센서를 가리지 않고
카메라에만 병을 보여 줍니다.

Next...

다음 시간엔?

NEXT

다음 시간 · 정지에서 회피로

오늘은 멈추는 조건을 만들었습니다

박스 위치와 회피

물체의 위치로
좌우 이동 방향을 정합니다.

직접 모은 데이터

기본 모델에 없는 대상을
새로운 라벨로 학습합니다.

표지판과 행동

인식 결과를
로버의 동작에 연결합니다.

인지 · 판단 · 제어가 이어지는 전체 루프를 확장합니다.

브릭 소스

app-bricks-py에서
카메라 루프와 콜백 읽기

OpenCV 직접 적용

Camera(adjustments=...)로
전처리 파이프라인 넣기

모델 바꾸기

Edge Impulse 모델과
EI_V_OBJ_DETECTION_MODEL

웹 화면 수정

assets/index.html, app.js
거리 그래프와 제어 UI

I2C 센서 2개

XSHUT으로 순서대로 깨워
겹치지 않는 주소 설정

리눅스 더 배우기

systemctl, journalctl, tmux
docker logs <컨테이너>

용어	쉬운 뜻	파트
MCU · MPU	정해진 제어를 빠르게 반복하는 칩 · 운영체제(리눅스)로 여러 일을 하는 프로세서	OPENING
모터 드라이버	MCU의 약한 신호를 받아 모터에 큰 전류를 대 주는 부품 (DRV8833)	OPENING
ToF	빛이 물체에 갔다 돌아오는 시간으로 거리를 재는 방식	PART 1
I2C · SDA · SCL	데이터선과 박자선 2개로 여러 장치와 주소를 붙여 대화하는 통신 규칙	PART 1
0x (16진수)	16진수로 쓴 숫자라는 표시. 0x29 = 41	PART 1
라이브러리	남이 만들어 둔 기능 묶음 (예: Adafruit VL53L0X)	PART 1
변수 · 함수	값을 담는 이름표 상자 · 이름을 붙여 둔 작업 묶음	PART 1
RTOS	정해진 시간 안에 반응하도록 만든 가벼운 운영체제 (MCU의 Zephyr)	PART 2
SRAM · LPDDR4	프로그램이 일하는 동안 데이터를 올려 두는 메모리의 종류	PART 2
GPIO	전기 신호를 내보내거나 읽는 범용 입출력 핀	PART 2
PWM	아주 빠르게 켜다 끄다 하는 비율로 모터 세기를 조절하는 방법	PART 2
RPC · Bridge	다른 프로세서의 함수를 이름으로 부르는 방법 · UNO Q의 두 칩 사이 통로	PART 2
콜백	어떤 일이 생기면 자동으로 불리도록 맡겨 둔 함수	PART 2
교착 상태	서로 상대의 응답을 기다리느라 둘 다 멈춘 상태	PART 2

용어	쉬운 뜻	파트
터미널 · 셸	글자 명령으로 컴퓨터를 다루는 창 · 입력한 명령을 해석해 실행하는 프로그램	PART 3
SSH	네트워크로 다른 컴퓨터의 터미널에 암호화해 접속하는 방법	PART 3
IP 주소 · 포트	네트워크 속 컴퓨터의 주소 · 그 컴퓨터 안 프로그램의 방 번호	PART 3
절대경로 · 상대경로	/ 부터 전부 적은 파일 위치 · 지금 있는 폴더를 기준으로 적은 위치	PART 3
장치 파일 · 드라이버	하드웨어를 파일처럼 여는 창구(/dev) · 하드웨어를 직접 다루는 운영체제 부품	PART 3
파이프 · 리다이렉션	앞 명령의 출력을 다음 명령으로 넘기기() · 출력을 파일로 저장(> , >>)	PART 3
프로세스 · 컨테이너	실행 중인 프로그램 · 프로그램과 실행 환경을 통째로 담은 상자 (Docker)	PART 3
픽셀 · 해상도 · FPS	이미지의 한 점 · 가로×세로 픽셀 수 · 1초에 처리하는 사진 장 수	PART 4
OpenCV	카메라 영상을 읽고 바꾸는 데 쓰는 영상 처리 라이브러리	PART 4
학습 · 추론	정답 달린 데이터로 모델을 만드는 과정 · 만든 모델로 새 입력에 답하는 과정	PART 4
YOLOX-Nano · COCO	한 번의 계산으로 여러 물체를 찾는 작은 검출 모델 · 80종 라벨의 공개 데이터셋	PART 4
신뢰도 · 임계값	모델이 매긴 점수 · 이 점수 이상만 검출로 인정하는 기준	PART 5
debounce	같은 신호가 너무 자주 반복 처리되지 않도록 간격을 두는 것	PART 5
JPEG · JSON · WebSocket	압축 이미지 형식 · '이름: 값' 텍스트 형식 · 계속 열려 있는 양방향 연결	PART 5